HIL-SERL — a visual walkthrough

Why HIL-SERL is the playbook's #1 RL method for SO-101 pick-and-place, what every moving part does, and exactly where the RL update happens.

1. What HIL-SERL is, in one sentence

HIL-SERL = **RLPD** (a sample-efficient off-policy RL algorithm) **+ a human watching with a gamepad**. The human's job is to intervene when the policy is about to make a costly mistake; those interventions become extra training data. The "RL" part is standard SAC — soft actor-critic — but it never has to explore catastrophically because the human is the safety net.

Why we care: it's the only RL method with a documented reproduction on SO-100/SO-101 hardware (ggando, ~70% on grasp-only after three weeks of debugging) and the only RL method with a first-class LeRobot port (`huggingface.co/docs/lerobot/hilserl`). The HIL-SERL paper itself (Luo et al. 2024, arXiv:2410.21845) reports **100% success on all 13 tasks in 1–2.5 h wall-clock** on a Franka arm.

2. All the components and how they wire together

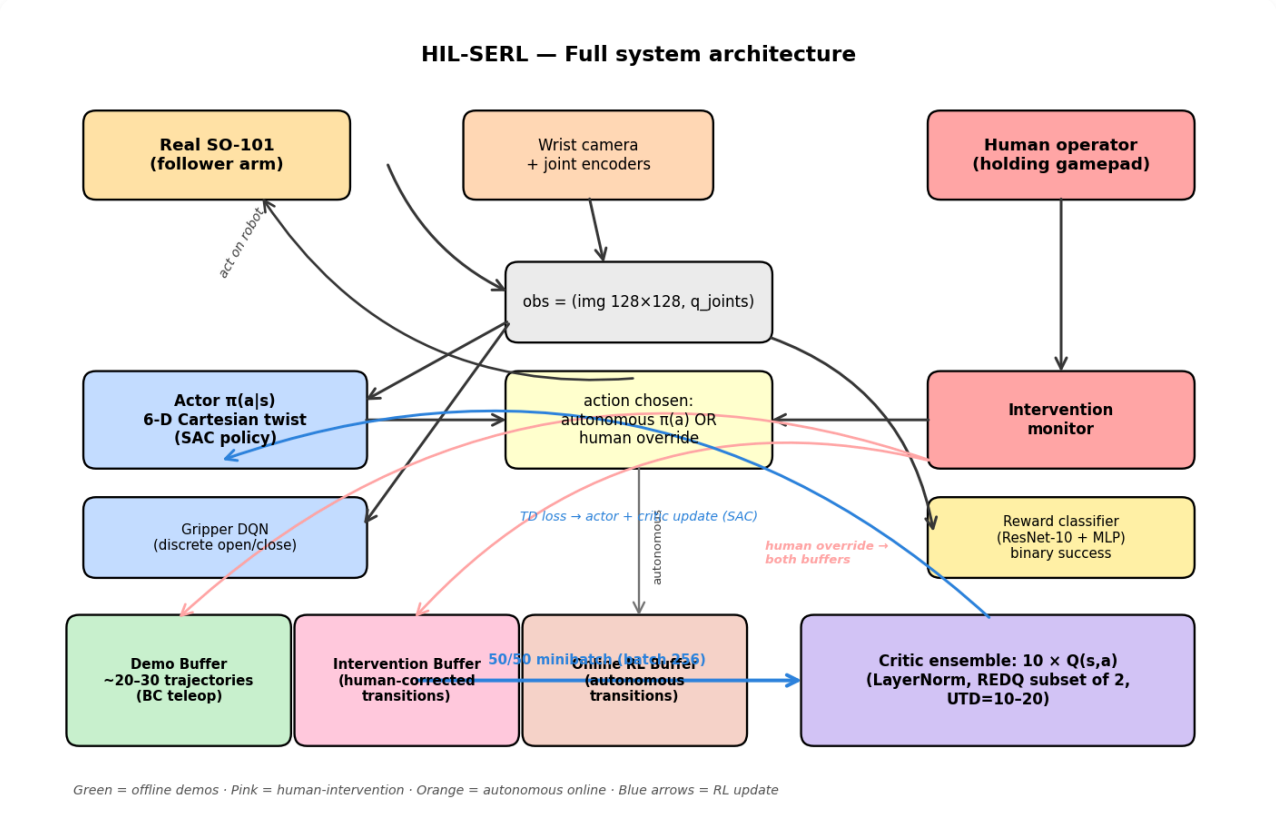

Figure 1 — every component and every data flow. Pink = human-in-the-loop; green = offline demos; orange = autonomous online experience; blue = RL update.

Component-by-component

Component	What it is	Concrete spec
Real robot	The SO-101 follower. Receives commanded actions, returns observations.	Wrist cam 128×128 RGB, joint encoders, 10 Hz control loop, impedance ctrl at 1 kHz with Δ-clip.
Actor π(a|s)	The policy. Maps observation → 6-D Cartesian twist action (continuous).	ResNet-10 (frozen, ImageNet) + 2-layer MLP, 256-wide. Initialized from your BC primitive (e.g. SmolVLA).
Gripper DQN	Separate policy for the discrete open/close action. Trained as its own MDP.	Small MLP, ε-greedy exploration.
Critic ensemble	10 Q-functions Q(s,a). REDQ subset of 2 used per update for pessimism.	Each: 2-layer MLP 256-wide, LayerNorm on every hidden layer. UTD=10–20.
Reward classifier	Trained binary success/failure detector. Sidesteps manual reward shaping.	ResNet-10 + MLP head, ~200 positive + ~1000 negative human-labeled frames, >95% acc.
Demo buffer	Pre-filled from teleop demonstrations. Never overwritten.	~20–30 trajectories. For us: a subset of our 101 episodes.
Intervention buffer	Stores human-corrected transitions. Routed to *both* demo and online buffers.	Grows during training. Per-transition (s, a_human, r, s').
Online RL buffer	Autonomous transitions from the actor acting in the environment.	Capped-size circular buffer. Per-transition (s, a_π, r, s').
Human + gamepad	You. Watching the robot. Pressing buttons when you see disaster coming.	Continuous monitoring, ~1–2 h wall-clock per task per the paper.

3. The 50/50 split — the heart of RLPD

Standard off-policy RL has one replay buffer. RLPD (the algorithm under HIL-SERL) has two — and every gradient update sees them in equal measure. This is the single most important design choice in the system.

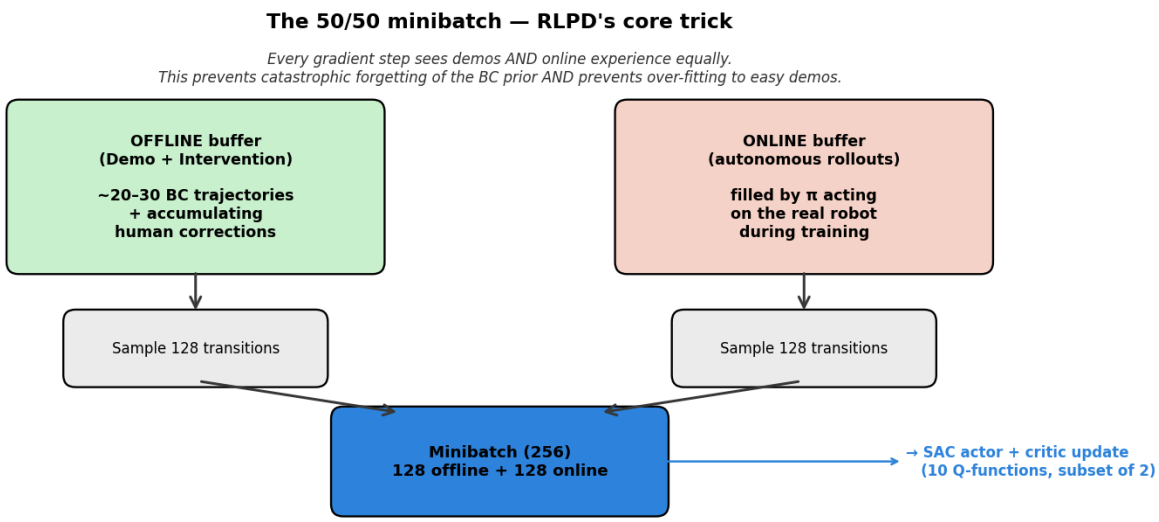

Figure 2 — Every minibatch (256) is exactly 128 offline + 128 online. The model never gets to "forget" the demos.

> **Why 50/50 and not, say, 90/10 demos?** If you over-weight demos, the policy never breaks free of imitating exactly what the human did, and you lose the RL benefits. If you under-weight demos, early online exploration is catastrophic because the actor doesn't know what "good behavior" looks like. 50/50 is the empirically validated balance from the RLPD paper (arXiv:2302.02948).

4. Where the "RL" actually happens — the SAC update

Everything above is plumbing. The *learning* happens in two equations, one for the critic and one for the actor, applied per minibatch.

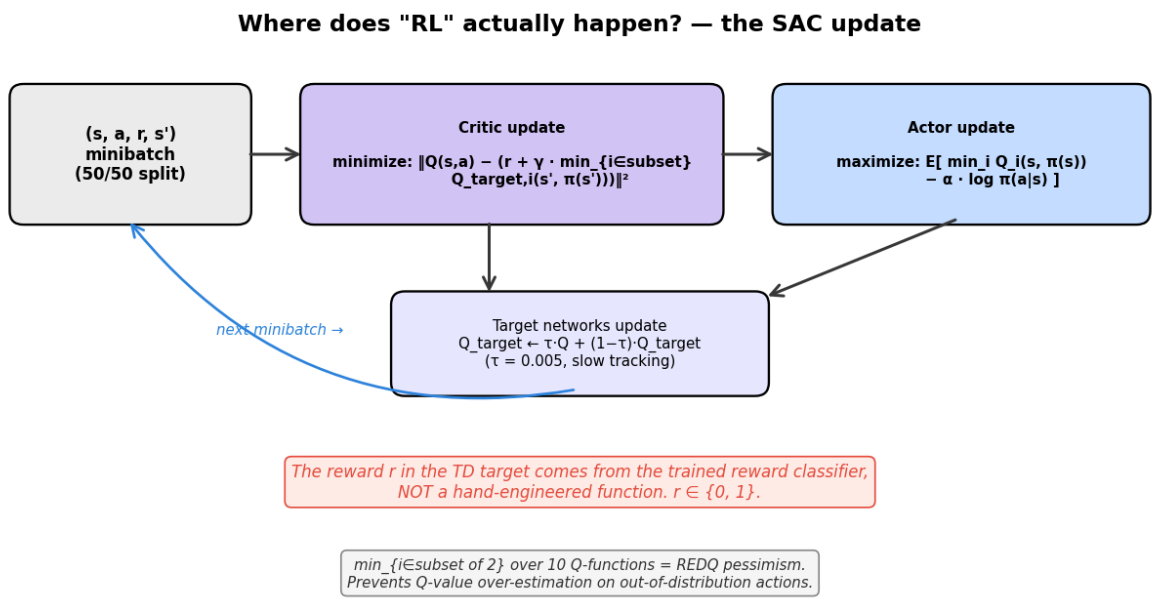

Figure 3 — The TD update (critic) and the policy improvement (actor). Both run per minibatch, multiple times per environment step (UTD = update-to-data ratio of 10–20).

In plain English:

- **Critic update:** "Adjust Q so it correctly predicts the reward you actually got plus the discounted Q of where you ended up." Standard temporal-difference learning. The `min` over a random subset of the 10 Q-functions is the REDQ pessimism trick — it prevents the policy from exploiting over-optimistic Q-values on actions it hasn't tried yet.
- **Actor update:** "Choose actions that maximize Q, but with an entropy bonus so you keep exploring." Standard SAC. The α entropy coefficient is auto-tuned (SAC auto-α).
- **Target update:** Slow-tracking copies of the Q-functions (τ=0.005) used to compute the TD target. Stops the bootstrap from chasing its own tail.

The **reward r** in those equations comes from the trained classifier, *not* a hand-designed dense reward. It's binary: 0 or 1. That's all the policy needs to know.

5. How the human-in-the-loop part actually works

At every step the human watches the rollout. If the policy is about to do something dumb — overshoot the grasp, knock the cube off the table, miss the bowl — the human grabs the gamepad and overrides the action. That single decision routes the transition into the right buffer(s).

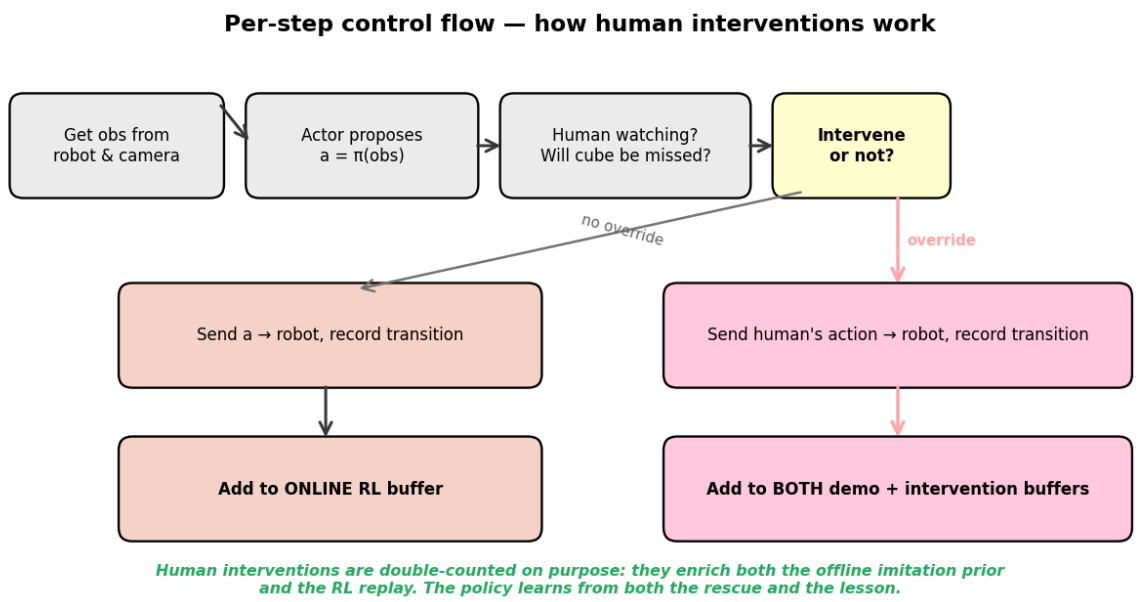

Figure 4 — Per-step decision tree. Autonomous transitions go to the online buffer only. Intervention transitions go to both the demo buffer AND the intervention buffer.

> **The double-write is the key mechanism.** Intervention transitions are valuable for two different reasons: as imitation targets (they show the actor what the right action *was*) and as reward signal (they're high-quality (s, a, r, s') tuples that the critic learns from). Routing them to both buffers means both networks see them on every update.

6. The reward classifier — how we get a learnable reward signal

Manual reward shaping for robot manipulation is notoriously difficult (we know from our PPO-from-scratch archive — see CLAUDE.md). HIL-SERL replaces this with a learned classifier:

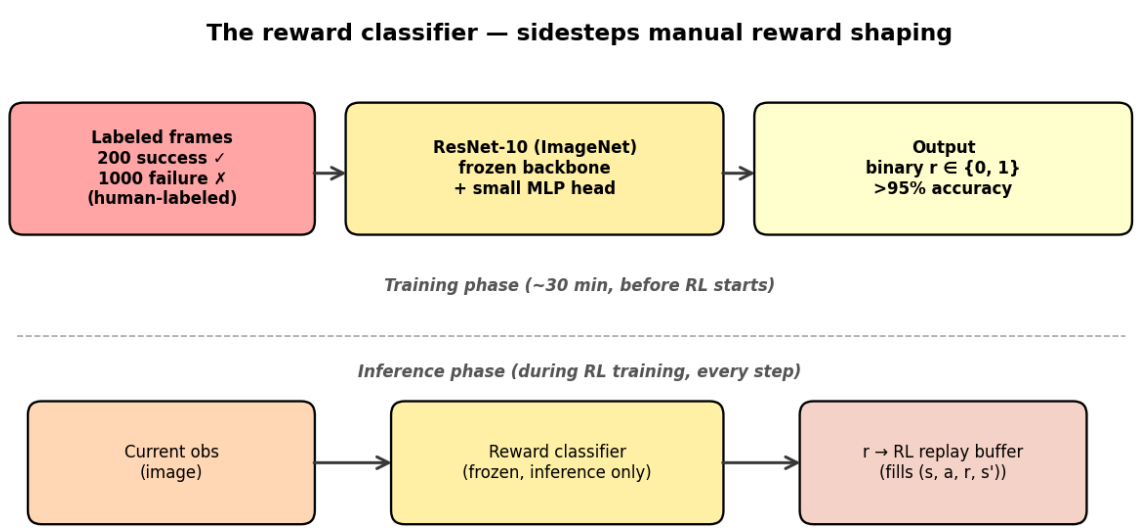

Figure 5 — Two-phase reward model. Train it once offline from labeled success/failure frames, then use it as a frozen black box during RL training.

Labeling is cheap: you watch your own teleop episodes and tag each frame as "success state" or "failure state." ~200 positives + ~1000 negatives gets you >95% accuracy. The frozen ResNet-10 backbone (pretrained on ImageNet) means you're effectively learning a small MLP on top of strong visual features — a few hundred examples is enough.

7. The training process, end-to-end

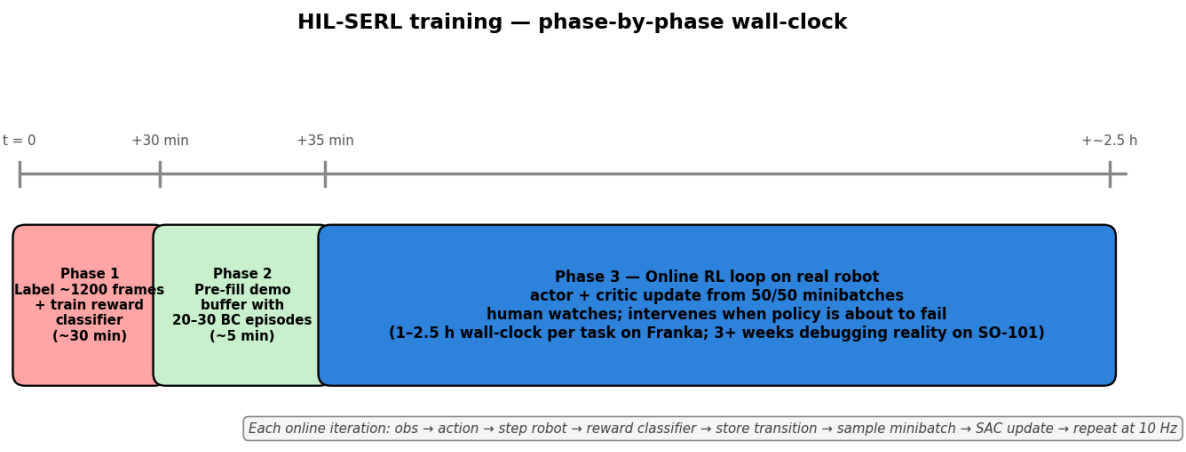

Figure 6 — Wall-clock view. The paper's 1–2.5 h is real on a Franka with a well-tuned setup. Add 3+ weeks of debugging on SO-101 specifically, per ggando's reproduction.

1. **Phase 1 (offline, ~30 min):** Label ~1200 frames from teleop episodes as success/failure. Train the reward classifier. Validate >95% accuracy on a held-out

set.

2. **Phase 2 (offline, ~5 min):** Pre-fill the demo buffer with 20–30 teleop trajectories. Initialize the actor from your BC primitive (e.g., the SmolVLA finetune from Week 1).
3. **Phase 3 (online, 1–2.5 h on Franka):** The RL loop runs at 10 Hz. The actor proposes actions. The reward classifier scores frames. You watch and intervene. The critic and actor update from 50/50 minibatches every step. UTD=10–20 means the networks update 10–20 times per environment step — this is what makes HIL-SERL sample-efficient enough for real hardware.

8. What to expect on SO-101 specifically

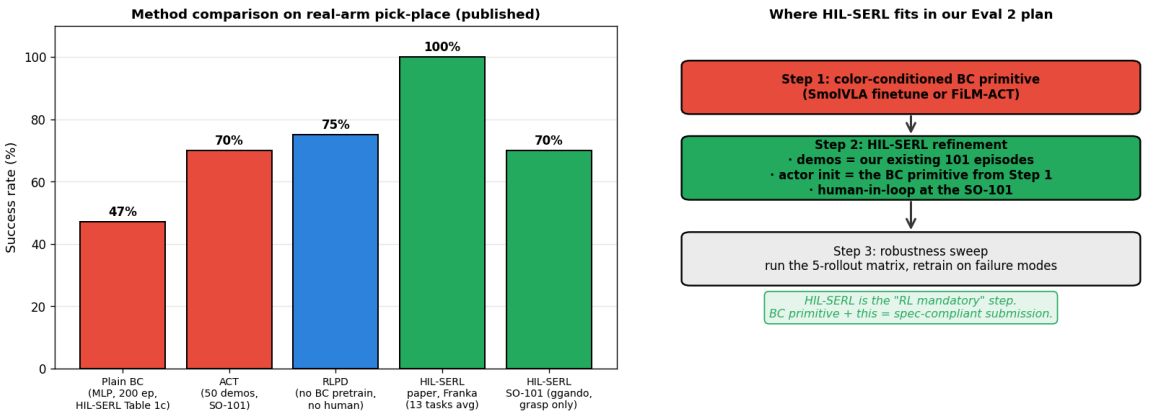

Figure 7 — Left: published numbers from low-cost arm reproductions. Right: how HIL-SERL fits into our Week 1–6 Eval 2 plan.

> **⚠ Reality check from the playbook:**
>
> - **LeRobot issue #1387:** "so101 can't work with HIL-SERL" — only `so100_follower_end_effector` works out of the box. Expect to need a port or fix.
> - **ggando reproduction:** reached ~70% on grasp-only after 3 weeks of fixes: MuJoCo-FK replacement, state caching bug, lighting sensitivity.
> - **Indraneel Patil:** *"performance is similar to IL with the same wall-clock"* — i.e., HIL-SERL doesn't beat well-tuned BC by huge margins on SO-101.
>
> Treat HIL-SERL as a robustness booster on top of a working BC primitive, not as a magic substitute for demonstration data.

9. Why this is the right RL choice for Eval 2 (the playbook's argument)

Property	What it means for us
Reuses our 101 teleop episodes	Demo buffer is pre-filled from our existing dataset. Nothing wasted.
Builds on a BC primitive	The SmolVLA (or FiLM-ACT) policy from Week 1 is the actor initialization. Cumulative wins.
Real-robot training (no sim-to-real gap)	No Isaac Lab / Blackwell sm_120 pain. We train on the actual SO-101 in the lab.
Human-in-loop safety net	You override before disasters happen. The policy never destroys the gripper, the cubes, or itself.
Reward classifier learnable	Skips the dense-reward design problem that killed our archived PPO-from-scratch attempt.
1st-class LeRobot port	We don't have to reimplement RLPD from scratch. The framework is there.
Satisfies "RL mandatory"	SAC + replay buffer + Q-learning is unambiguously RL by the spec's intent.

Generated by `notes/visualizations/generate_hilserl_walkthrough.py` · references `notes/so101_robot_learning_playbook.md` §T2, §3, §5 + HIL-SERL paper arXiv:2410.21845